



UNIVERSIDADE DO VALE DO ITAJAÍ

ESCOLA POLITÉCNICA

CIÊNCIAS DA COMPUTAÇÃO

Relatório M1: IPC, Threads e Paralelismo

Atividade prática M1

Aquilis Adrian

Filipe Dietrich

Nilton Tadeu

Ciência da Computação - Sistemas Operacionais

Univali, Campus Itajaí, 2026

1. Enunciado do Projeto

O projeto consiste no desenvolvimento de um sistema que simula o funcionamento interno de um gerenciador de requisições a um banco de dados. O sistema deve utilizar múltiplos processos e threads para realizar as seguintes tarefas:

- Um processo cliente envia requisições de consulta ou inserção via IPC (Pipe ou memória compartilhada).
- Um processo servidor recebe as requisições e utiliza threads para processá-las em paralelo.
- As threads utilizam mecanismos de sincronização (mutex ou semáforo) para garantir acesso seguro ao banco de dados simulado.

2. Explicação e Contexto da Aplicação

A solução foi desenvolvida em C++ utilizando a API do Windows para a comunicação entre processos e a biblioteca padrão de threads para o paralelismo.

Componentes:

1. **IPC (Inter-Process Communication):** Utilizou-se **Pipes Nominais** (*Named Pipes*) para a comunicação bidirecional entre o cliente e o servidor.
2. **Servidor Gerenciador:** O servidor cria o pipe e aguarda conexões. Para cada mensagem recebida, ele despacha uma thread independente, permitindo que múltiplas requisições sejam processadas sem bloquear o recebimento de novos dados.
3. **Banco de Dados Simulado:** Os dados são persistidos em um arquivo de texto (banco.txt). As operações suportadas são CRUD completo: INSERT, SELECT, UPDATE e DELETE.
4. **Sincronização:** Como o arquivo é um recurso compartilhado entre as threads, foi implementado um semáforo binário. Este mecanismo garante que apenas uma thread por vez realize operações de leitura e escrita no arquivo, evitando condições de corrida e corrupção de dados.

3. Resultados Obtidos com as Simulações

Para validar a robustez do sistema, foi executado o programa cliente_automacao.cpp, que simula uma carga de trabalho automatizada:

- **Paralelismo:** O console do servidor registrou múltiplos *IDs de Thread* operando simultaneamente.

- **Integridade:** Mesmo com requisições rápidas e sucessivas, o semáforo garantiu que as operações de INSERT e UPDATE fossem atômicas em relação ao arquivo físico.
- **Tratamento de Erros:** O sistema identificou corretamente tentativas de inserção de IDs duplicados e buscas por registros inexistentes.

4. Códigos Importantes de Implementação

Estrutura da Requisição (banco.h)

```
typedef struct {
    Operacao op;
    Registro dados;
} Requisicao;
```

Criação de Threads no Servidor (Servidor.cpp)

```
while (ReadFile(hPipe, &req, sizeof(Requisicao), &bytesRead, NULL) && bytesRead > 0) {
    // Cria a THREAD para processar em paralelo
    thread t(processar_requisicao, req);
    t.detach(); // Permite que a thread rode independentemente
}
```

Controle de Concorrência com Semáforo (Servidor.cpp)

```
binary_semaphore semaforo_banco{ 1 };
void processar_requisicao(Requisicao req) {
    semaforo_banco.acquire(); // Bloqueia acesso de outras threads
    // Lógica de manipulação do banco.txt
    semaforo_banco.release(); // Libera para a próxima thread
}
```

5. Resultados da Implementação

Abaixo, a tabela de operações realizadas durante a bateria de testes automatizados:

Operação	Entrada (ID)	Nome/Parâmetro	Resultado no Servidor
INSERT	1 a 5	Usuário_Original_X	Sucesso: Registros criados
SELECT	3	-	Sucesso: Registro 3 localizado

Operação	Entrada (ID)	Nome/Parâmetro	Resultado no Servidor
UPDATE	2	NOME_ATUALIZADO	Sucesso: Nome alterado no arquivo
DELETE	1	-	Sucesso: Registro removido
INSERT	3	TENTATIVA_DUPLICADA	Erro: ID já existente detectado

6. Análise e Discussão

A arquitetura baseada em threads e IPC mostrou-se eficiente para o problema proposto. O uso de **Named Pipes** facilitou a troca de estruturas complexas entre processos distintos. A principal discussão técnica recai sobre o uso do semáforo: embora ele garanta a consistência, ele serializa o acesso ao arquivo de texto. Em um sistema de produção, poderiam ser usados mecanismos como mutex para permitir leituras (SELECT) simultâneas, bloqueando o recurso apenas em escritas (INSERT/UPDATE/DELETE). No entanto, para os objetivos desta avaliação, a solução protege adequadamente a seção crítica.